Production-Ready CI/CD and Kubernetes Infrastructure for Java Microservices Payment Application

This repository contains the complete codebase and documentation for a production-ready Continuous Integration/Continuous Delivery (CI/CD) pipeline and Kubernetes infrastructure designed for a Java microservices-based payment application. The solution is tailored to serve 10 million global customers, emphasizing scalability, reliability, security, and observability.

Table of Contents

1. Project Overview
2. Microservices Architecture
3. CI/CD Pipeline with GitHub Actions
    - SonarQube Integration
    - Trivy Container Scanning
4. Containerization with Docker
5. Kubernetes and vCluster Setup
    - vCluster Creation Guide
    - Namespace Strategy
6. GitOps with ArgoCD
7. Kubernetes Manifests with Helm
8. Scalability, Load Balancing, and SSL
9. Monitoring and Logging Stack
    - Grafana Dashboards and Alerts
    - Slack Integration
10. Production Readiness Checklist
11. Repository Structure
12. Setup and Deployment Guide

1. Project Overview

This project provides an end-to-end solution for deploying and managing a Java microservices payment application on Kubernetes. It includes:

- **Six Java Microservices**: Sample Spring Boot applications.
- **GitHub Actions**: Automated CI/CD for building, testing, scanning, and pushing Docker images.
- **SonarQube**: Static code analysis for quality and security.
- **Trivy**: Container image vulnerability scanning.
- **Docker**: Containerization of microservices.
- **Kubernetes**: Orchestration platform for deployment.
- **vCluster**: Virtual Kubernetes clusters for isolated environments.
- **ArgoCD**: GitOps-driven continuous deployment.
- **Helm**: Package manager for Kubernetes applications.
- **Autoscaling**: Horizontal Pod Autoscalers (HPA) for dynamic scaling.
- **Load Balancing**: Ingress controllers for external traffic management.
- **SSL Certificates**: Automated provisioning with Cert-Manager.
- **Monitoring**: Grafana for visualization and alerting.
- **Logging**: Filebeat, Logstash, and Elasticsearch (ELK stack components) for log collection and processing.
- **Alerting**: Grafana alerts integrated with Slack.

2. Microservices Architecture

The payment application is composed of six independent Java microservices, each responsible for a specific business capability. While the provided code includes basic Spring Boot applications, in a real-world scenario, these would be fully developed services with dedicated business logic and data stores.

Example Microservices:

- `user-service`: Manages user profiles and authentication.
- `payment-gateway-service`: Handles payment processing and integration with external payment providers.
- `transaction-service`: Records and manages payment transactions.
- `fraud-detection-service`: Analyzes transactions for suspicious activities.
- `notification-service`: Sends user notifications (e.g., email, SMS) for transaction updates.

- `reporting-service` : Generates reports on payment activities.

Each microservice is designed to be stateless (where possible) and communicate primarily via RESTful APIs. Asynchronous communication patterns (e.g., using Kafka or RabbitMQ) can be integrated for event-driven workflows, enhancing resilience and scalability.

3. CI/CD Pipeline with GitHub Actions

The CI/CD pipeline is orchestrated using GitHub Actions, providing automated workflows for building, testing, and deploying the microservices. The `main.yml` workflow defines the following stages:

- **Code Checkout**: Fetches the source code from the GitHub repository.
- **Build & Test**: Compiles Java code using Maven and runs unit tests.
- **Code Quality (SonarQube)**: Performs static code analysis to identify bugs, vulnerabilities, and code smells. Integration requires `SONAR_TOKEN` and `SONAR_HOST_URL` as GitHub Secrets.
- **Containerization**: Builds Docker images for each microservice using multi-stage Dockerfiles.
- **Vulnerability Scan (Trivy)**: Scans Docker images for known vulnerabilities. Critical and high-severity vulnerabilities will fail the build.
- **Image Push**: Pushes the built Docker images to a container registry (e.g., Docker Hub). Requires `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN` as GitHub Secrets.
- **GitOps Trigger**: After successful image push, the workflow updates the image tags in the respective Helm `values.yaml` files within a separate GitOps repository, triggering ArgoCD for deployment.

GitHub Actions Workflow (`.github/workflows/main.yml`):

```yaml
# ... (content of .github/workflows/main.yml)
```

SonarQube Integration

SonarQube is integrated into the CI pipeline to maintain high code quality standards. Each microservice's code is scanned, and results are reported to a SonarQube instance. This helps in early detection of issues and ensures adherence to coding best practices.

Trivy Container Scanning

Trivy is used to scan Docker images for operating system packages and application dependencies vulnerabilities. This step is crucial for supply chain security, ensuring that deployed containers are free from known security flaws.

4. Containerization with Docker

Each microservice is containerized using Docker. A generic multi-stage `Dockerfile.template` is used to build optimized, lightweight images. This approach ensures consistency across all microservices and reduces the attack surface of the final images.

Dockerfile Structure (`microservices/Dockerfile.template`):

```
Plain Text

# ... (content of microservices/Dockerfile.template)
```

For local development and testing, a `docker-compose.yml` file is provided, allowing developers to spin up all microservices and their dependencies with a single command.

Docker Compose (`microservices/docker-compose.yml`):

```
YAML

# ... (content of microservices/docker-compose.yml)
```

5. Kubernetes and vCluster Setup

The microservices are deployed on a Kubernetes cluster. To provide isolated environments, `vCluster` is utilized. A vCluster acts as a lightweight, virtual Kubernetes cluster running on top of a larger host Kubernetes cluster. This is ideal for creating development, staging, or even production environments with strong isolation without the overhead of managing multiple full-fledged clusters.

vCluster Creation Guide

To set up a vCluster, you would typically use the `vcluster` CLI. Below are the general steps:

1. **Install vCluster CLI**: Follow the official vCluster documentation to install the CLI tool.
2. **Create a vCluster**: Use the `vcluster create` command, optionally specifying a `values.yaml` file for configuration.

   ```
   Bash
   ```

```
vcluster create my-payment-vcluster -n vcluster-payment --distro k3s --expos
```

The `vcluster-values.yaml` provided in this repository (`vcluster/vcluster-values.yaml`) configures resource limits, synchronization settings, and isolation policies for the vCluster control plane.

```yaml
# ... (content of vcluster/vcluster-values.yaml)
```

3. **Connect to vCluster**: Once created, you can connect to your vCluster using:

```bash
vcluster connect my-payment-vcluster -n vcluster-payment
```

This command updates your `kubeconfig` to point to the vCluster, allowing you to interact with it as if it were a standalone cluster.

Namespace Strategy

Within the vCluster, a clear namespace strategy is employed for better organization, resource management, and access control:

- `payment-app` : Dedicated namespace for all 6 microservices of the payment application.
- `monitoring` : Namespace for monitoring components like Grafana and Prometheus.
- `logging` : Namespace for logging components like Logstash, Filebeat, and Elasticsearch.

This separation ensures that different components of the infrastructure do not interfere with each other and simplifies resource quotas and network policies.

Namespace Definitions (`vcluster/namespaces.yaml`):

```yaml
# ... (content of vcluster/namespaces.yaml)
```

6. GitOps with ArgoCD

ArgoCD is used to implement GitOps, ensuring that the desired state of the Kubernetes cluster is declared in Git and automatically synchronized. This provides a single source of truth for deployments, enabling faster, more reliable, and auditable releases.

- **GitOps Repository**: A dedicated Git repository (e.g., `gitops-repo`) will store all Kubernetes manifests, including Helm charts and ArgoCD Application definitions.
- **ArgoCD ApplicationSet**: An `ApplicationSet` is used to manage the deployment of multiple microservices. It dynamically generates ArgoCD `Application` resources based on a list of services, pointing to their respective Helm charts in the GitOps repository.

ArgoCD ApplicationSet (`argocd/applicationset.yaml`):

```yaml
# ... (content of argocd/applicationset.yaml)
```

This `ApplicationSet` will monitor the `charts/{{service}}` path in the `gitops-repo` for changes, automatically deploying or updating the corresponding microservice in the `payment-app` namespace.

7. Kubernetes Manifests with Helm

Each microservice is deployed using its own Helm chart, providing a templated and configurable way to define Kubernetes resources. A generic `microservice-template` chart is provided and then customized for each of the six microservices.

Common Helm Chart Components:

- **Deployment**: Manages the desired state of the microservice pods.
- **Service**: Exposes the microservice within the cluster.
- **Ingress**: Manages external access to the microservice, integrating with load balancers and SSL.
- **Horizontal Pod Autoscaler (HPA)**: Automatically scales the number of pods based on CPU/memory utilization or custom metrics.
- **ServiceAccount, Role, RoleBinding**: Configures Kubernetes Role-Based Access Control (RBAC).
- **ConfigMaps and Secrets**: Manages configuration and sensitive data.

Helm Chart Structure (e.g., `charts/user-service/`):

```plain text
charts/
├── user-service/
│   ├── Chart.yaml
│   ├── values.yaml
│   └── templates/
```

```
│       ├── _helpers.tpl
│       ├── deployment.yaml
│       ├── hpa.yaml
│       ├── ingress.yaml
│       ├── service.yaml
│       └── serviceaccount.yaml
├── payment-gateway-service/
│   ├── ...
└── ... (other microservices)
```

8. Scalability, Load Balancing, and SSL

To handle 10 million global customers, the infrastructure incorporates robust solutions for scalability, load balancing, and security.

- **Autoscaling**: Horizontal Pod Autoscalers (HPA) are configured in each microservice's Helm chart (`hpa.yaml`). They automatically adjust the number of pod replicas based on CPU and memory utilization. For advanced scenarios, custom metrics (e.g., message queue length, request latency) can be integrated.
- **Load Balancing**: An NGINX Ingress Controller is deployed to manage external traffic. It integrates with cloud provider-specific load balancers (e.g., AWS ALB, GCP Load Balancer) to distribute incoming requests across microservice instances. The `ingress-controller-values.yaml` provides an example configuration for AWS NLB integration.

  Ingress Controller Configuration (`infrastructure/ingress-controller-values.yaml`):

  ```yaml
  # ... (content of infrastructure/ingress-controller-values.yaml)
  ```

- **SSL Certificates**: Cert-Manager is used for automated provisioning and renewal of SSL/TLS certificates from Let's Encrypt. A `ClusterIssuer` (`infrastructure/cluster-issuer.yaml`) is defined to handle certificate requests for all ingresses.

  Cert-Manager ClusterIssuer (`infrastructure/cluster-issuer.yaml`):

  ```yaml
  # ... (content of infrastructure/cluster-issuer.yaml)
  ```

9. Monitoring and Logging Stack

A comprehensive monitoring and logging solution is critical for a production payment application. This setup leverages the ELK stack (Elasticsearch, Logstash, Kibana - though Kibana is not explicitly included in this setup, Grafana can query Elasticsearch) and Prometheus/Grafana.

- **Metrics Collection**: Prometheus (not explicitly configured here, but assumed as the data source for Grafana) collects metrics from Kubernetes and application endpoints.
- **Log Collection (Filebeat)**: Filebeat is deployed as a `DaemonSet` (`logging/filebeat/filebeat-daemonset.yaml`) on each node to collect container logs and forward them to Logstash. Alternatively, Filebeat can be deployed as a sidecar container within each microservice pod for more granular log collection.

  Filebeat DaemonSet (`logging/filebeat/filebeat-daemonset.yaml`):

  ```yaml
  # ... (content of logging/filebeat/filebeat-daemonset.yaml)
  ```

- **Log Processing (Logstash)**: Logstash receives logs from Filebeat, filters, parses, and enriches them before sending them to Elasticsearch for storage and indexing. A sample `logstash-config.yaml` is provided for basic parsing.

  Logstash Configuration (`logging/logstash/logstash-config.yaml`):

  ```yaml
  # ... (content of logging/logstash/logstash-config.yaml)
  ```

- **Monitoring & Visualization (Grafana)**: Grafana is deployed as a `Deployment` (`monitoring/grafana/grafana-deployment.yaml`) and provides dashboards for visualizing metrics and logs. It connects to Prometheus for metrics and Elasticsearch for logs.

  Grafana Deployment (`monitoring/grafana/grafana-deployment.yaml`):

  ```yaml
  # ... (content of monitoring/grafana/grafana-deployment.yaml)
  ```

Grafana Dashboards and Alerts

A sample Grafana dashboard (`monitoring/grafana/dashboard.json`) is included to monitor key metrics such as CPU usage per pod and internal server error rates (5xx errors) per service. This dashboard provides immediate visibility into the application's health and performance.

Grafana Dashboard (`monitoring/grafana/dashboard.json`):

```JSON
// ... (content of monitoring/grafana/dashboard.json)
```

Grafana Alerting is configured to send notifications based on predefined rules. The `alerts.yaml` defines example alert rules for high CPU usage and high 5xx error rates.

Grafana Alert Rules (`monitoring/grafana/alerts.yaml`):

```YAML
# ... (content of monitoring/grafana/alerts.yaml)
```

Slack Integration

Alerts generated by Grafana are integrated with Slack to notify the operations team in real-time. The `alerts.yaml` includes a placeholder for a Slack webhook URL, which needs to be replaced with your actual Slack incoming webhook.

10. Production Readiness Checklist

To ensure the application is production-ready, the following considerations have been incorporated:

- **Resource Limits & Requests**: Defined for all containers in Helm charts to prevent resource exhaustion and ensure fair sharing.
- **Liveness & Readiness Probes**: Configured for all microservices to enable Kubernetes to manage application health and traffic routing effectively.
- **Rolling Updates**: Kubernetes deployments are configured for rolling updates by default, ensuring zero-downtime deployments.
- **Secrets Management**: Kubernetes Secrets are used for sensitive information. For enhanced security, integration with external secret management solutions (e.g., HashiCorp Vault, AWS Secrets Manager) is recommended.
- **Backup & Restore**: A robust backup and restore strategy for databases and persistent volumes is crucial and should be implemented based on the chosen data stores.
- **Global Distribution**: For 10 million global customers, consider multi-region deployments with global DNS (e.g., AWS Route 53, Cloudflare) for low latency and disaster recovery.

11. Repository Structure

Plain Text

```
.
├── .github/
│   └── workflows/
│       └── main.yml               # GitHub Actions CI/CD workflow
├── argocd/
│   └── applicationset.yaml        # ArgoCD ApplicationSet for microservices
├── charts/
│   ├── microservice-template/     # Helm chart template
│   │   ├── Chart.yaml
│   │   ├── values.yaml
│   │   └── templates/
│   │       ├── _helpers.tpl
│   │       ├── deployment.yaml
│   │       ├── hpa.yaml
│   │       ├── ingress.yaml
│   │       ├── service.yaml
│   │       └── serviceaccount.yaml
│   ├── user-service/              # Helm chart for user-service
│   ├── payment-gateway-service/   # Helm chart for payment-gateway-service
│   ├── transaction-service/       # Helm chart for transaction-service
│   ├── fraud-detection-service/   # Helm chart for fraud-detection-service
│   ├── notification-service/      # Helm chart for notification-service
│   └── reporting-service/         # Helm chart for reporting-service
├── infrastructure/
│   ├── cluster-issuer.yaml        # Cert-Manager ClusterIssuer
│   └── ingress-controller-values.yaml # NGINX Ingress Controller values
├── logging/
│   ├── filebeat/
│   │   └── filebeat-daemonset.yaml # Filebeat DaemonSet
│   └── logstash/
│       └── logstash-config.yaml    # Logstash configuration
├── microservices/
│   ├── user-service/              # Java user-service
│   │   ├── pom.xml
│   │   ├── Dockerfile
│   │   └── src/main/java/com/example/userservice/UserServiceApplication.java
│   ├── payment-gateway-service/   # Java payment-gateway-service
│   │   ├── pom.xml
│   │   ├── Dockerfile
│   │   └── src/main/java/com/example/paymentgatewayservice/PaymentGatewayServi
│   ├── transaction-service/       # Java transaction-service
│   │   ├── pom.xml
│   │   ├── Dockerfile
│   │   └── src/main/java/com/example/transactionservice/TransactionServiceAppl
│   ├── fraud-detection-service/   # Java fraud-detection-service
```

```
│   │   ├── pom.xml
│   │   ├── Dockerfile
│   │   └── src/main/java/com/example/frauddetectionservice/FraudDetectionServi
│   ├── notification-service/      # Java notification-service
│   │   ├── pom.xml
│   │   ├── Dockerfile
│   │   └── src/main/java/com/example/notificationservice/NotificationServiceAp
│   ├── reporting-service/         # Java reporting-service
│   │   ├── pom.xml
│   │   ├── Dockerfile
│   │   └── src/main/java/com/example/reportingservice/ReportingServiceApplicat
│   ├── Dockerfile.template        # Generic Dockerfile template
│   └── docker-compose.yml         # Docker Compose for local development
├── monitoring/
│   └── grafana/
│       ├── alerts.yaml            # Grafana alert rules
│       └── dashboard.json         # Grafana dashboard JSON
├── vcluster/
│   ├── namespaces.yaml            # Kubernetes namespaces for vCluster
│   └── vcluster-values.yaml       # vCluster configuration values
└── README.md                      # This documentation
```

12. Setup and Deployment Guide

This section provides a high-level guide to setting up and deploying the payment application using the provided codebase.

Prerequisites

- A GitHub account.
- A Docker Hub account (or other container registry).
- A Kubernetes host cluster (e.g., EKS, GKE, AKS, or a self-managed Kubeadm cluster).
- `kubectl` configured to access your host cluster.
- `helm` CLI installed.
- `vcluster` CLI installed.
- `argocd` CLI installed.

Step-by-Step Deployment

1. **Fork this Repository**: Fork this GitHub repository to your own GitHub account.
2. **Create GitHub Secrets**: In your forked repository, go to `Settings > Secrets and variables > Actions` and add the following repository secrets:

- `SONAR_TOKEN` : Your SonarQube authentication token.
- `SONAR_HOST_URL` : The URL of your SonarQube instance.
- `DOCKERHUB_USERNAME` : Your Docker Hub username.
- `DOCKERHUB_TOKEN` : Your Docker Hub access token.
- `GITOPS_REPO_TOKEN` : A GitHub Personal Access Token (PAT) with `repo` scope to allow GitHub Actions to push changes to your GitOps repository.

3. **Create GitOps Repository**: Create a new, empty GitHub repository (e.g., `gitops-repo`) in your account. This repository will store the Kubernetes manifests that ArgoCD will synchronize.
4. **Initial Push to GitOps Repo**: Copy the `charts/` directory from this repository into your `gitops-repo` and push it to the `main` branch. This will be the source for ArgoCD.
5. **Install vCluster**: On your host Kubernetes cluster, install vCluster using the provided `vcluster-values.yaml` :

   Bash
   ```bash
   helm upgrade --install my-payment-vcluster vcluster/vcluster --repo https://
   ```

6. **Connect to vCluster**: Connect to your newly created vCluster:

   Bash
   ```bash
   vcluster connect my-payment-vcluster -n vcluster-payment
   ```

7. **Apply Namespaces**: Apply the namespace definitions to your vCluster:

   Bash
   ```bash
   kubectl apply -f vcluster/namespaces.yaml
   ```

8. **Install Cert-Manager**: Install Cert-Manager on your vCluster (or host cluster if preferred for cluster-wide resources):

   Bash
   ```bash
   helm repo add jetstack https://charts.jetstack.io
   helm repo update
   helm upgrade --install cert-manager jetstack/cert-manager \
     --namespace cert-manager --create-namespace \
   ```

```
  --version v1.13.0 \
  --set installCRDs=true
```

9. **Apply ClusterIssuer**: Apply the `ClusterIssuer` for Let's Encrypt:

```bash
kubectl apply -f infrastructure/cluster-issuer.yaml
```

10. **Install NGINX Ingress Controller**: Install the NGINX Ingress Controller on your vCluster using the provided values:

```bash
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm upgrade --install ingress-nginx ingress-nginx/ingress-nginx \
  --namespace ingress-nginx --create-namespace \
  -f infrastructure/ingress-controller-values.yaml
```

11. **Install ArgoCD**: Install ArgoCD on your vCluster (or host cluster if preferred for cluster-wide resources):

```bash
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-c
```

12. **Configure ArgoCD ApplicationSet**: Update the `argocd/applicationset.yaml` with your GitHub username for the `repoURL` and apply it to your vCluster:

```bash
# Replace {{github_username}} with your actual GitHub username in argocd/app
sed -i 's/{{github_username}}/YOUR_GITHUB_USERNAME/g' argocd/applicationset.
kubectl apply -f argocd/applicationset.yaml
```

13. **Install Monitoring and Logging Components**:
    - **Elasticsearch**: Install Elasticsearch (e.g., using a Helm chart from Elastic or ECK operator) in the `logging` namespace.
    - **Logstash**: Deploy Logstash using the provided configuration:

      ```bash
      ```

```
kubectl apply -f logging/logstash/logstash-config.yaml
# You will need a Deployment/StatefulSet for Logstash itself, referencing
# Example (simplified):
# kubectl apply -f logging/logstash/logstash-deployment.yaml # (Not provi
```

- **Filebeat**: Deploy Filebeat as a DaemonSet:

  ```bash
  kubectl apply -f logging/filebeat/filebeat-daemonset.yaml
  # Ensure a ServiceAccount and ClusterRole/ClusterRoleBinding are created
  ```

- **Grafana**: Deploy Grafana using the provided configuration. Ensure a PersistentVolumeClaim named `grafana-pvc` exists or is created.

  ```bash
  kubectl apply -f monitoring/grafana/grafana-deployment.yaml
  # You will also need a Service and Ingress for Grafana access.
  ```

- **Apply Grafana Dashboard and Alerts**: Once Grafana is running, you can import the dashboard and configure alert rules. This can be done via the Grafana UI or by using Grafana's provisioning features.

  ```bash
  # Manual import via UI or use Grafana Provisioning for dashboards and ale
  # Update the Slack webhook URL in monitoring/grafana/alerts.yaml before c
  ```

14. **Trigger CI/CD**: Push changes to your microservice code in your forked repository. This will trigger the GitHub Actions pipeline, which will build, scan, and push images, then update the GitOps repository. ArgoCD will then automatically deploy the updated microservices to your vCluster.

This comprehensive setup provides a robust, scalable, and observable foundation for your Java microservices payment application.